

Core CS - OOPS



Core CS Interview Prep — OOPs



Structure

Subject	Top 10	Top 25	Top 50
OOPs	Most asked, high-priority	Covers core concepts	Deep dive

OOPS:

→ **Top 10:**

OOPS: Top 10

1. What is Object-Oriented Programming and what are its four pillars?

OOP is a programming paradigm that organizes code around objects rather than functions. The four pillars are Encapsulation (bundling data and methods, hiding internals), Abstraction (exposing only what's necessary), Inheritance (a class deriving properties from another), and Polymorphism (same interface, different behavior).

2. What is the difference between a class and an object?

A class is a blueprint — it defines structure and behavior. An object is an instance of that class, created at runtime with actual values in memory.

3. What is Encapsulation and why does it matter?

Encapsulation restricts direct access to an object's internal state using access modifiers (private, protected, public). It protects data integrity and reduces unintended side effects, making code easier to maintain.

4. What is Abstraction? How is it different from Encapsulation?

Abstraction hides *what* is complex by exposing only relevant features (via abstract classes or interfaces). Encapsulation hides *how* the internals work. Abstraction is about design; encapsulation is about implementation.

5. What is Inheritance? What problem does it solve?

Inheritance allows a child class to reuse properties and methods of a parent class, promoting code reuse and establishing an "is-a" relationship. It reduces duplication and makes hierarchical modeling natural.

6. What is Polymorphism? Explain compile-time vs runtime.

Polymorphism means one interface, many forms. Compile-time (static) polymorphism is achieved through method overloading — same method name, different parameters. Runtime (dynamic) polymorphism is through method overriding — a child class redefines a parent method, and the correct one is called based on the actual object type at runtime.

7. What is the difference between method overloading and method overriding?

Overloading is within the same class — same method name, different signatures, resolved at compile time. Overriding is across parent-child classes — same signature redefined in the child, resolved at runtime via dynamic dispatch.

8. What is an abstract class vs an interface?

An abstract class can have both abstract and concrete methods, and can hold state. An interface (traditionally) defines only method contracts with no implementation. A class can inherit from only one abstract class but implement multiple interfaces. Use abstract classes for shared base behavior, interfaces for defining capabilities.

9. What is the difference between association, aggregation, and composition?

All three represent relationships between objects. Association is a general "uses-a" link. Aggregation is a "has-a" relationship where the child can exist independently (e.g., a department and its employees). Composition is a stronger "has-a" where the child cannot exist without the parent (e.g., a house and its rooms).

10. What are the SOLID principles?

Five design principles for maintainable OOP code: **S**ingle Responsibility (a class does one thing), **O**pen/Closed (open for extension, closed for modification), **L**iskov Substitution (subclasses should be substitutable for their parent), **I**nterface Segregation (prefer many specific interfaces over one general one), **D**ependency Inversion (depend on abstractions, not concretions).

→ **Top 25:**



OOPS — Top 25 Interview Questions

1. What is Object-Oriented Programming?

A programming paradigm that organizes code around objects — instances of classes — rather than functions and logic. It models real-world entities and their interactions.

2. What are the four pillars of OOP?

Encapsulation, Abstraction, Inheritance, and Polymorphism. Each addresses a different aspect of how objects are designed and interact.

3. What is a Class vs an Object?

A **class** is a blueprint — it defines attributes and behaviors. An **object** is an instance of that class with actual values assigned to those attributes.

4. What is Encapsulation?

Bundling data (fields) and methods that operate on that data into a single unit (class), and restricting direct access to some of the object's components using access modifiers (`private`, `protected`, `public`).

5. What is Abstraction?

Hiding implementation details and exposing only the necessary interface to the user. Achieved via abstract classes and interfaces. Reduces complexity and increases reusability.

6. What is Inheritance?

A mechanism where a child class acquires properties and behaviors of a parent class. Promotes code reuse. Types: single, multilevel, hierarchical, multiple (via interfaces in Java/C#).

7. What is Polymorphism?

The ability of one interface to be used for different underlying data types. Two types:

- **Compile-time (static):** Method overloading
- **Runtime (dynamic):** Method overriding

8. What is Method Overloading?

Defining multiple methods in the same class with the **same name but different parameter lists** (type, count, or order). Resolved at compile time.

9. What is Method Overriding?

A child class provides a specific implementation of a method already defined in its parent class, using the **same signature**. Resolved at runtime via dynamic dispatch.

10. What is the difference between an Abstract Class and an Interface?

Abstract Class	Interface
Can have concrete methods	Only abstract methods (pre-Java 8)
Single inheritance	Multiple inheritance supported
Can have constructors	Cannot have constructors
Used for "is-a" relationship	Used for "can-do" capability contracts

11. What is a Constructor?

A special method called automatically when an object is created. It initializes the object's state. It has the same name as the class and no return type.

12. What is Constructor Overloading?

Defining multiple constructors in a class with different parameter lists, allowing objects to be created in different ways.

13. What is the `this` keyword?

Refers to the **current instance** of the class. Used to differentiate between instance variables and parameters with the same name, and to call other constructors within the same class.

14. What is the `super` keyword?

Refers to the **parent class**. Used to call the parent class's constructor or methods from within a child class, especially when they've been overridden.

15. What is a Static method vs an Instance method?

- **Static:** Belongs to the class, not to any object. Called without creating an instance. Cannot access instance variables.

- **Instance:** Belongs to an object. Can access and modify instance variables.
-

16. What is the difference between Composition and Inheritance?

- **Inheritance:** "is-a" relationship. Child class extends parent.
- **Composition:** "has-a" relationship. A class contains a reference to another class as a field.

Composition is generally preferred for flexibility and to avoid tight coupling.

17. What is Coupling and Cohesion?

- **Coupling:** Degree of dependency between classes. Low coupling is desirable.
 - **Cohesion:** Degree to which elements within a class belong together. High cohesion is desirable.
-

18. What is the SOLID principle?

A set of five design principles:

- **S** — Single Responsibility
 - **O** — Open/Closed
 - **L** — Liskov Substitution
 - **I** — Interface Segregation
 - **D** — Dependency Inversion
-

19. What is the Liskov Substitution Principle?

Objects of a subclass should be replaceable with objects of the superclass without breaking the program. The child class must honor the contract defined by the parent.

20. What is an Access Modifier?

Keywords that control the visibility of classes, methods, and fields:

- `private` → class only
 - `protected` → class + subclasses
 - `public` → everywhere
 - `default/package` → same package only
-

21. What is a Destructor / Finalizer?

A method called when an object is about to be destroyed to release resources. In Java, this is `finalize()` (deprecated). In C++, it's `~ClassName()`. In Python, it's `__del__`.

22. What is an Association, Aggregation, and Composition?

- **Association:** General relationship between two classes.
 - **Aggregation:** "has-a" relationship where the child can exist independently of the parent.
 - **Composition:** Strong "has-a" where the child's lifecycle depends on the parent.
-

23. What is a Virtual Function?

A function declared in a base class that can be overridden in derived classes. Enables **runtime polymorphism**. In C++, declared with the `virtual` keyword. In Java, all non-static methods are virtual by default.

24. What is an Anonymous Class?

A class defined and instantiated in a single expression, without a name. Commonly used to provide a one-time implementation of an interface or abstract class, often in event handling or callbacks.

25. What is the difference between early binding and late binding?

- **Early binding (static):** Method call resolved at **compile time**. Applies to overloaded methods and `final` / `static` methods.
- **Late binding (dynamic):** Method call resolved at **runtime** based on the actual object type. Applies to overridden methods and enables runtime polymorphism.

→ **Top 50:**



OOPS — Top 50 Interview Questions & Answers

1. What is Object-Oriented Programming (OOP)?

OOP is a programming paradigm that organizes software design around objects — instances of classes — rather than functions and logic. It models real-world entities using four core principles: Encapsulation, Abstraction, Inheritance, and Polymorphism.

2. What are the four pillars of OOP?

- **Encapsulation** – Bundling data and methods that operate on it within a class, restricting direct external access.
 - **Abstraction** – Hiding implementation details and exposing only the necessary interface.
 - **Inheritance** – A class (child) acquiring properties and behaviors of another class (parent).
 - **Polymorphism** – The ability of a single interface to represent different underlying forms (method overloading/overriding).
-

3. What is a Class?

A class is a blueprint or template that defines the attributes (fields) and behaviors (methods) common to all objects of that type. It does not occupy memory until an object is instantiated from it.

4. What is an Object?

An object is an instance of a class. It occupies memory, holds actual values for the class's attributes, and can invoke the class's methods. Every object has identity, state, and behavior.

5. What is Encapsulation?

Encapsulation is the practice of wrapping data (variables) and methods that manipulate that data within a single unit (class), and restricting direct access to internal state via access modifiers (`private` , `protected` , `public`). It promotes data hiding and reduces unintended interference.

6. What is Abstraction?

Abstraction means exposing only the relevant features of an object while hiding the internal implementation. It is achieved through abstract classes and interfaces. Example: A car's steering wheel abstracts the underlying steering mechanism.

7. What is Inheritance?

Inheritance allows a child class to inherit properties and methods from a parent class, enabling code reuse. It establishes an **IS-A** relationship. Example: `Dog` is a `Animal`.

8. What is Polymorphism?

Polymorphism means "many forms." It allows the same method name to behave differently based on the object or arguments:

- **Compile-time (Static):** Method Overloading
 - **Runtime (Dynamic):** Method Overriding
-

9. What is the difference between Method Overloading and Method Overriding?

Aspect	Overloading	Overriding
When resolved	Compile-time	Runtime
Where	Same class	Parent-child classes
Signature	Different parameters	Same signature
Return type	Can differ	Must be same (or covariant)

10. What is a Constructor?

A constructor is a special method invoked automatically when an object is created. It has the same name as the class and no return type. Its purpose is to initialize the object's state.

11. What are the types of Constructors?

- **Default Constructor** – No parameters; provided by the compiler if none is defined.

- **Parameterized Constructor** – Accepts arguments to initialize fields with specific values.
- **Copy Constructor** – Creates a new object as a copy of an existing object (common in C++).

12. What is a Destructor?

A destructor is a method called when an object is destroyed or goes out of scope. It releases resources (memory, file handles, etc.) held by the object. In Java/C#, the garbage collector handles this; in C++ it's explicitly defined with

```
~ClassName() .
```

13. What is the difference between an Abstract Class and an Interface?

Aspect	Abstract Class	Interface
Instantiation	Cannot be instantiated	Cannot be instantiated
Methods	Can have concrete + abstract	Only abstract (pre-Java 8); default methods from Java 8+
Variables	Can have instance variables	Only <code>public static final</code>
Inheritance	Single inheritance	Multiple inheritance supported
Use case	Shared base with partial impl.	Define a contract/capability

14. What is the difference between `this` and `super`?

- `this` refers to the current class instance — used to access current class fields, methods, or constructors.
- `super` refers to the parent class — used to access parent class fields, methods, or constructors from a subclass.

15. What is a Static method vs an Instance method?

- **Static method** belongs to the class, not to any object. It can be called without creating an instance and cannot access instance variables.

- **Instance method** belongs to an object. It can access and modify the object's state.
-

16. What is the difference between `==` and `.equals()` in Java?

- `==` checks reference equality — whether two references point to the same object in memory.
 - `.equals()` checks logical/value equality — whether two objects are meaningfully the same. It should be overridden in custom classes.
-

17. What is Coupling?

Coupling refers to the degree of dependency between classes or modules.

Tight coupling means classes are highly dependent on each other (bad for maintainability). **Loose coupling** means minimal dependencies (preferred — easier to test and modify).

18. What is Cohesion?

Cohesion refers to how closely related the responsibilities of a single class are.

High cohesion means a class does one thing well (preferred). **Low cohesion** means a class handles many unrelated responsibilities (anti-pattern).

19. What is the `final` keyword in Java?

- `final` **variable** – Value cannot be changed (constant).
 - `final` **method** – Cannot be overridden by a subclass.
 - `final` **class** – Cannot be subclassed (e.g., `String` in Java).
-

20. What is an Abstract Class?

An abstract class is a class declared with the `abstract` keyword that cannot be instantiated directly. It can contain both abstract methods (no body) and concrete methods (with body). Subclasses must implement all abstract methods.

21. What is an Interface?

An interface is a contract that defines a set of method signatures a class must implement. It supports multiple inheritance in Java and decouples what a class does from how it does it.

22. What is Multiple Inheritance and why doesn't Java support it with classes?

Multiple inheritance allows a class to inherit from more than one parent class. Java avoids it with classes to prevent the **Diamond Problem** — ambiguity when two parent classes have the same method. Java resolves this by supporting multiple inheritance through interfaces only.

23. What is the Diamond Problem?

When class **D** inherits from both **B** and **C**, and both **B** and **C** inherit from **A**, it's unclear which version of **A**'s method **D** should use. This ambiguity is called the Diamond Problem.

24. What is Composition vs Inheritance?

- **Inheritance** is an IS-A relationship (a **Dog** is an **Animal**).
- **Composition** is a HAS-A relationship (a **Car** has an **Engine**).

Composition is generally preferred over inheritance as it provides greater flexibility and avoids tight coupling.

25. What is Method Hiding?

In Java, if a child class defines a **static** method with the same signature as a parent's static method, it **hides** the parent method rather than overriding it. The method that gets called depends on the reference type, not the object type.

26. What is Runtime Polymorphism?

Runtime polymorphism (dynamic dispatch) occurs when an overridden method is resolved at runtime based on the actual object type, not the reference type. It is achieved through method overriding and upcasting.

27. What is Upcasting and Downcasting?

- **Upcasting** – Assigning a child class object to a parent class reference. Done implicitly. Safe.
 - **Downcasting** – Casting a parent reference back to a child type. Done explicitly. Can throw `ClassCastException` if invalid; use `instanceof` check before doing it.
-

28. What is the `instanceof` operator?

`instanceof` checks whether an object is an instance of a specific class or interface at runtime. It returns `true` or `false` and is commonly used before downcasting to avoid `ClassCastException`.

29. What is an Inner Class?

An inner class is a class defined within another class. It has access to the outer class's members, including private ones. Types in Java:

- **Static nested class**
 - **Non-static inner class**
 - **Local class** (inside a method)
 - **Anonymous class** (no name, declared and instantiated at once)
-

30. What is an Anonymous Class?

An anonymous class is a class without a name that is defined and instantiated in a single expression. It is commonly used for one-time implementations of interfaces or abstract classes, especially in event handling.

31. What is a Singleton Class?

A Singleton ensures only one instance of a class is created throughout the application. It is implemented by making the constructor `private`, providing a `static` method to return the single instance, and using a `static` variable to hold it.

32. What is Operator Overloading?

Operator overloading allows operators (like `+`, `-`) to have different behaviors depending on the operands. Supported in C++ and Python, but **not in Java** (except for `+` with `String`, which is handled by the compiler).

33. What is a Virtual Function?

A virtual function (in C++) is a function declared in a base class that can be overridden in derived classes. It enables runtime polymorphism. Java methods are virtual by default (except `static`, `final`, and `private`).

34. What is a Pure Virtual Function?

A pure virtual function in C++ is declared with `= 0` and has no implementation in the base class. A class containing at least one pure virtual function becomes an abstract class and cannot be instantiated.

35. What is a Friend Function in C++?

A friend function is a non-member function granted access to a class's private and protected members. It is declared inside the class with the `friend` keyword. It is not a member of the class and does not have a `this` pointer.

36. What is the difference between Compile-time and Runtime Polymorphism?

Aspect	Compile-time	Runtime
Also called	Static polymorphism	Dynamic polymorphism
Achieved via	Method overloading	Method overriding
Resolved at	Compile time	Runtime
Performance	Faster	Slightly slower (dynamic dispatch)

37. What is Association, Aggregation, and Composition?

- **Association** – A general relationship between two classes (uses-a). No ownership.
- **Aggregation** – A weak HAS-A relationship. Child can exist independently of the parent. (e.g., `Department` has `Professors`; professors exist without the

department.)

- **Composition** – A strong HAS-A relationship. Child cannot exist without the parent. (e.g., `House` has `Rooms` ; rooms don't exist without the house.)

38. What is the difference between an Object and a Class?

Class	Object
Blueprint/template	Instance of a class
Defined once	Created multiple times
No memory until object is made	Occupies memory

39. What is `super()` in constructors?

`super()` is used to call the parent class's constructor from within the child class's constructor. It must be the first statement in the child constructor. If not explicitly written, the compiler inserts `super()` automatically (calling the parent's no-arg constructor).

40. What is Constructor Chaining?

Constructor chaining is calling one constructor from another within the same class (using `this()`) or from a parent class (using `super()`). It reduces code duplication when multiple constructors share initialization logic.

41. What is an Immutable Class?

An immutable class is one whose state cannot be changed after creation. To make a class immutable:

- Declare it `final`
- Make all fields `private final`
- No setters
- Return copies of mutable fields in getters
- Example: `String` , `Integer` in Java.

42. What is the SOLID Principle?

SOLID is a set of five OOP design principles:

- **S** – Single Responsibility: A class should have one reason to change.
- **O** – Open/Closed: Open for extension, closed for modification.
- **L** – Liskov Substitution: Subtypes must be substitutable for their base types.
- **I** – Interface Segregation: Prefer small, specific interfaces over large ones.
- **D** – Dependency Inversion: Depend on abstractions, not concrete implementations.

43. What is the difference between `private`, `protected`, and `public` ?

Modifier	Same Class	Same Package	Subclass	Everywhere
<code>private</code>	✓	✗	✗	✗
<code>default</code>	✓	✓	✗	✗
<code>protected</code>	✓	✓	✓	✗
<code>public</code>	✓	✓	✓	✓

44. What is Garbage Collection?

Garbage collection is the automatic process of reclaiming memory occupied by objects that are no longer referenced. Java's JVM handles this automatically. The `finalize()` method (deprecated in Java 9+) was called before GC, but explicit resource cleanup should use `try-with-resources`.

45. What is the difference between Shallow Copy and Deep Copy?

- **Shallow Copy** – Copies the object's reference values. Nested objects are shared, not duplicated. Changes in nested objects affect both copies.
- **Deep Copy** – Recursively copies all nested objects. The copy is completely independent of the original.

46. What is a Marker Interface?

A marker interface is an empty interface (no methods or fields) used to signal a capability or metadata to the JVM or framework. Examples in Java:

`Serializable`, `Cloneable`. They are largely replaced by annotations in modern Java.

47. What is the difference between `new` and `newInstance()` ?

- `new` – Compile-time mechanism. Directly calls a constructor. Most common approach.
 - `newInstance()` – Runtime mechanism via Reflection
(`Class.forName("...").newInstance()`). Used when the class name isn't known at compile time. Requires the class to have a no-arg constructor.
-

48. What is Object Cloning?

Object cloning creates an exact copy of an object using the `clone()` method (from `Object` class). The class must implement the `Cloneable` marker interface, otherwise `CloneNotSupportedException` is thrown. The default `clone()` performs a shallow copy.

49. What is a Design Pattern? Name a few OOP-related ones.

A design pattern is a reusable solution to a commonly occurring problem in software design. Key OOP patterns:

- **Creational:** Singleton, Factory, Abstract Factory, Builder
 - **Structural:** Adapter, Decorator, Facade, Proxy
 - **Behavioral:** Observer, Strategy, Command, Template Method
-

50. What is the difference between Abstraction and Encapsulation?

Abstraction	Encapsulation
Hides implementation details	Hides internal data/state
Focuses on what an object does	Focuses on how it protects its data
Achieved via abstract classes/interfaces	Achieved via access modifiers
Design-level concept	Implementation-level concept